

Sol coding benchmark

Six programming tasks. Three models. Six reasoning effort levels.

Sol 5.6, Sol 6 and Sol 6.1 at Low, Medium, High, Xhigh, Max and Ultra. Two independent attempts per task, Standard speed.

ATTEMPTS 216 3 models x 6 efforts x 6 tasks x 2 repeats	API-EQUIVALENT TOTAL $\geq$ \$87.59 Observed token price; $\geq$ marks a lower bound	ACCEPTED 213/216 Completion and full functional acceptance
--	---	---

What the results show

Best measured cost with full acceptance: Sol 6.1 / low: \$0.101 per attempt, 161.7 seconds, 12/12 accepted. This is also the fastest setting with full acceptance.

Complex tasks: 71/72 accepted; 1115/1116 hidden checks passed. One Sol 6 Low optimizer attempt exceeded the 8-second scale limit. Every optimizer small-case oracle and all MVCC checks passed.

Broader behavioral evidence: 920/922 expanded checks passed on completed solutions for the first four tasks. Two Low cache solutions fail a valid huge-integer TTL boundary. 214/216 attempts completed; the two earlier noncompletions remain included in delivery and cost accounting.

Own-test evidence: 175/214 completed candidates have a usable score, including zero for no discovered added tests. 29 have no recognized added tests. The two new tasks have 55/72 usable scores. Manual rubric scoring remains incomplete.

Test-strength separation: Sol 6.1 MVCC added tests detect all six fixed defects in both repetitions at High, Xhigh, Max and Ultra. Optimizer test-strength evidence is often inconclusive, so this benefit is specific to the MVCC defect set.

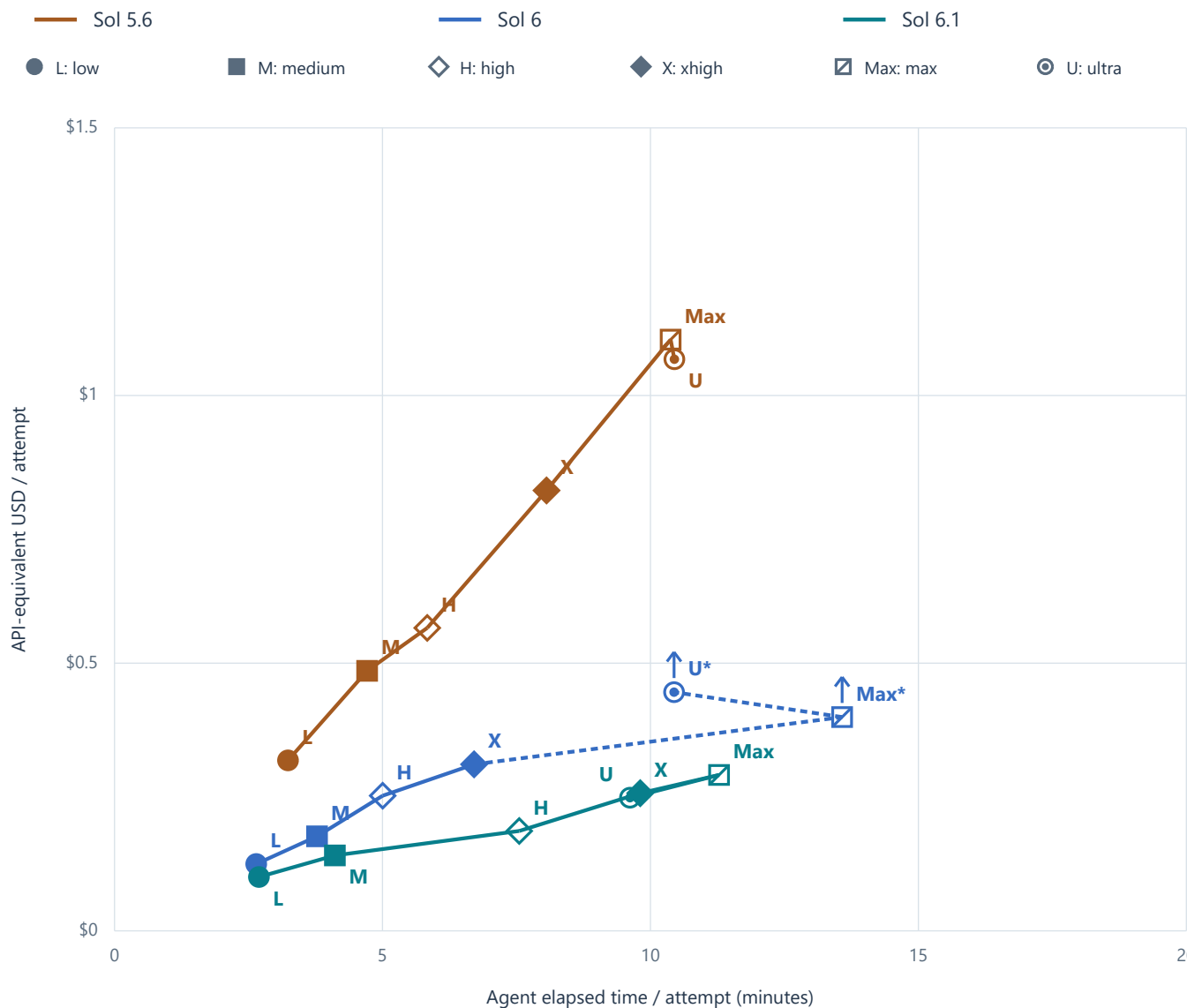
How to interpret the comparison

The lower-cost settings already solve nearly all tested behavior. Higher effort generally costs more and takes longer, with little additional correctness separation here. The complex tasks still show a strong score ceiling: this evidence supports a workload-specific comparison, without a reliable ranking of general intelligence.

USD is a counterfactual Standard API token estimate for observed Codex usage, not an invoice. Partial usage creates lower bounds. All report charts prioritize dollars and agent elapsed time; effort points are connected within each model.

Cost versus elapsed time

Each point averages twelve attempts: two repetitions of six equally weighted tasks. Lower cost and shorter time occupy the lower-left region.



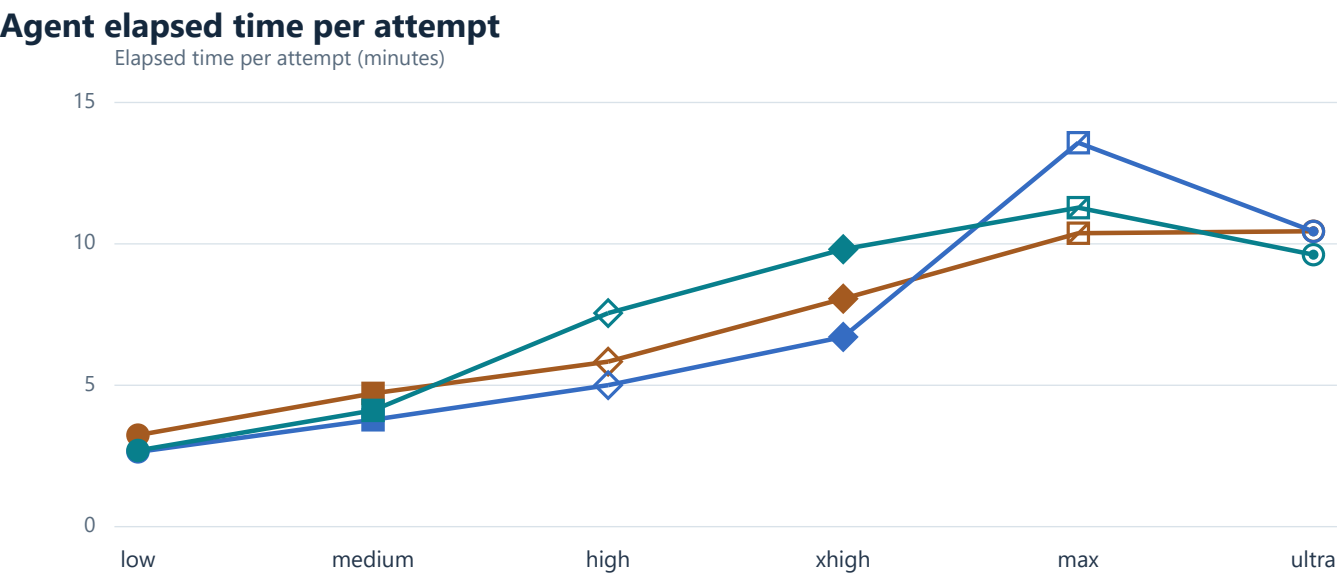
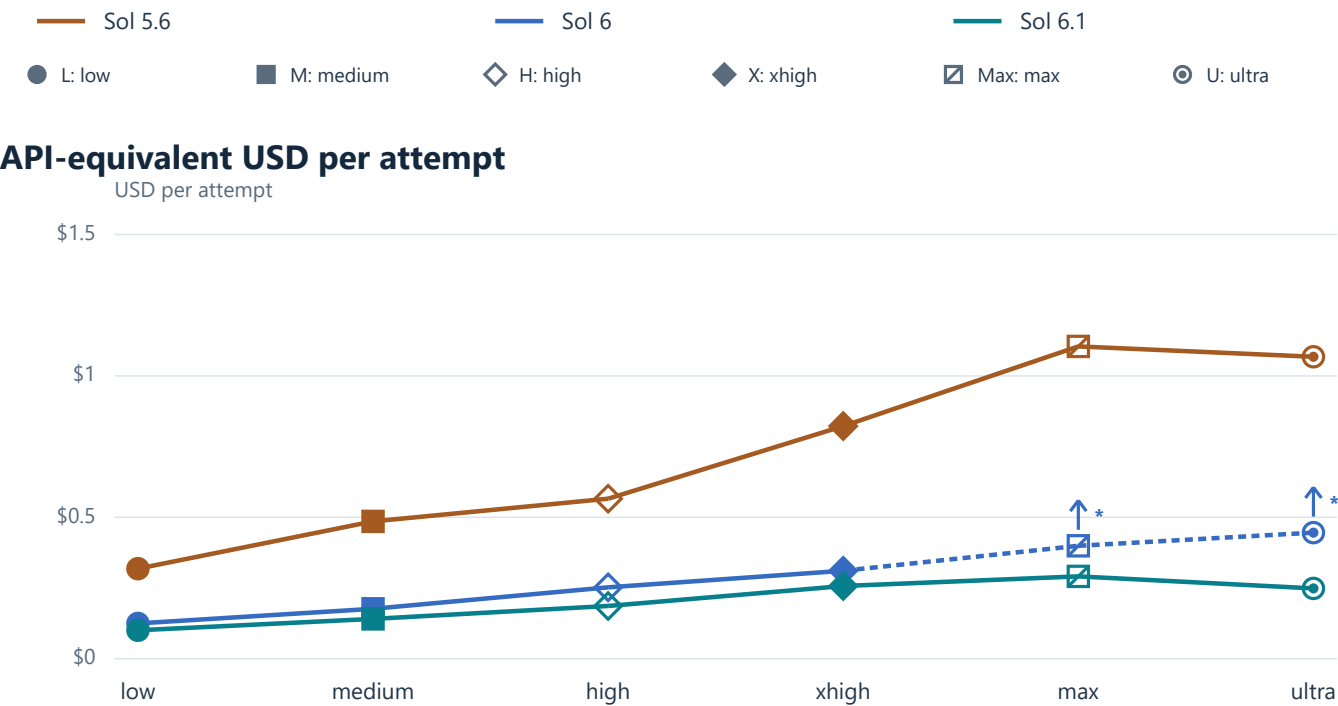
Labels: L = low, M = medium, H = high, X = xhigh, Max = max, U = ultra. Lines connect effort order. Stars, dashed links and upward arrows mark observed USD lower bounds where final usage is incomplete.

Agent elapsed time includes reasoning, tool calls and self-tests. Setup and external grading are excluded. Interrupted attempts retain their elapsed time and observed token cost. This is a full programming-workflow measurement.

The connected USD/time presentation is consistent with model benchmark cost-efficiency comparisons, including Artificial Analysis. The tasks and scoring scale are specific to this experiment.

Effort curves: money and time

Follow each model across six settings. Dashed USD segments use partial observations.



Higher effort is a setting, not a guarantee of higher correctness. Compare these premiums with the behavior and acceptance tables.

Dollars, time and acceptance

Twelve attempts per setting. USD / accepted includes spending on unsuccessful attempts.

Model / effort	USD / attempt	USD / accepted	Mean time, s	Accepted
Sol 5.6 / low	\$0.318	\$0.318	194.2	12/12
Sol 5.6 / medium	\$0.486	\$0.486	282.8	12/12
Sol 5.6 / high	\$0.566	\$0.566	350.1	12/12
Sol 5.6 / xhigh	\$0.823	\$0.823	483.7	12/12
Sol 5.6 / max	\$1.104	\$1.104	622.6	12/12
Sol 5.6 / ultra	\$1.068	\$1.068	626.8	12/12
Sol 6 / low	\$0.125	\$0.136	158.7	11/12
Sol 6 / medium	\$0.177	\$0.177	226.7	12/12
Sol 6 / high	\$0.252	\$0.252	300.2	12/12
Sol 6 / xhigh	\$0.311	\$0.311	402.6	12/12
Sol 6 / max	≥ \$0.399*	N/A	814.6	11/12
Sol 6 / ultra	≥ \$0.445*	N/A	626.4	11/12
Sol 6.1 / low	\$0.101	\$0.101	161.7	12/12
Sol 6.1 / medium	\$0.141	\$0.141	246.7	12/12
Sol 6.1 / high	\$0.187	\$0.187	453.1	12/12
Sol 6.1 / xhigh	\$0.257	\$0.257	588.6	12/12
Sol 6.1 / max	\$0.291	\$0.291	676.8	12/12
Sol 6.1 / ultra	\$0.249	\$0.249	577.1	12/12

Sol 6 / Max has a 40-minute reservation-service timeout; its saved code passes the functional and expanded checks. Sol 6 / Ultra has a provider capacity failure on the interval task. Sol 6 / Low completes every attempt but misses one optimizer scale check. These are distinct outcomes.

N/A in accepted-cost columns means final total spending is unavailable. Displayed >= costs are observed lower bounds, rounded down.

What does extra effort buy?

Multiples use the same model at Low as the baseline. Functional quality is an equal six-task macro average.

Model / effort	Cost multiple	Time multiple	Functional / 100	Accepted
Sol 5.6 / low	1.00	1.00	100.00	12/12
Sol 5.6 / medium	1.53	1.46	100.00	12/12
Sol 5.6 / high	1.78	1.80	100.00	12/12
Sol 5.6 / xhigh	2.58	2.49	100.00	12/12
Sol 5.6 / max	3.47	3.21	100.00	12/12
Sol 5.6 / ultra	3.35	3.23	100.00	12/12
Sol 6 / low	1.00	1.00	99.44	11/12
Sol 6 / medium	1.41	1.43	100.00	12/12
Sol 6 / high	2.02	1.89	100.00	12/12
Sol 6 / xhigh	2.49	2.54	100.00	12/12
Sol 6 / max	N/A	5.13	100.00	11/12
Sol 6 / ultra	N/A	3.95	91.67	11/12
Sol 6.1 / low	1.00	1.00	100.00	12/12
Sol 6.1 / medium	1.40	1.53	100.00	12/12
Sol 6.1 / high	1.85	2.80	100.00	12/12
Sol 6.1 / xhigh	2.56	3.64	100.00	12/12
Sol 6.1 / max	2.90	4.19	100.00	12/12
Sol 6.1 / ultra	2.47	3.57	100.00	12/12

The main score averages the two repetitions within each task, then gives each of six tasks equal weight. It includes retained snapshot grades; acceptance also requires completed delivery. The capacity-failed stub scores zero; the timeout snapshot has passing code but is not accepted delivery.

Do not read a small score difference as a statistically established effect. Each model/effort/task has only two observations.

Workload trajectories: core tasks

Each task point averages two attempts. Axis ranges differ between panels.



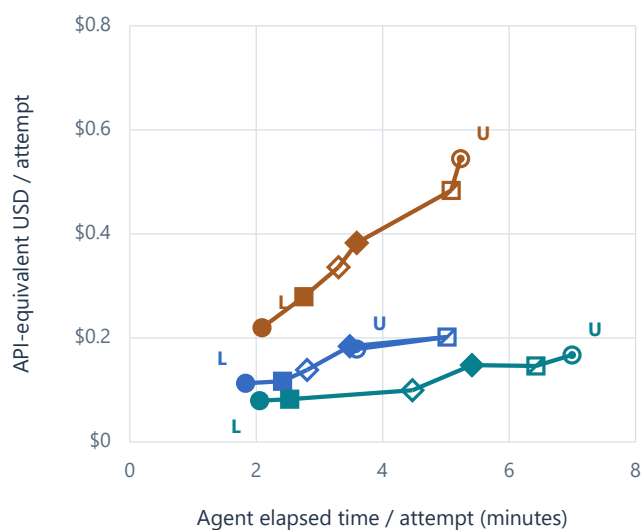
Intervals



TTL/LRU cache



Concurrent DAG



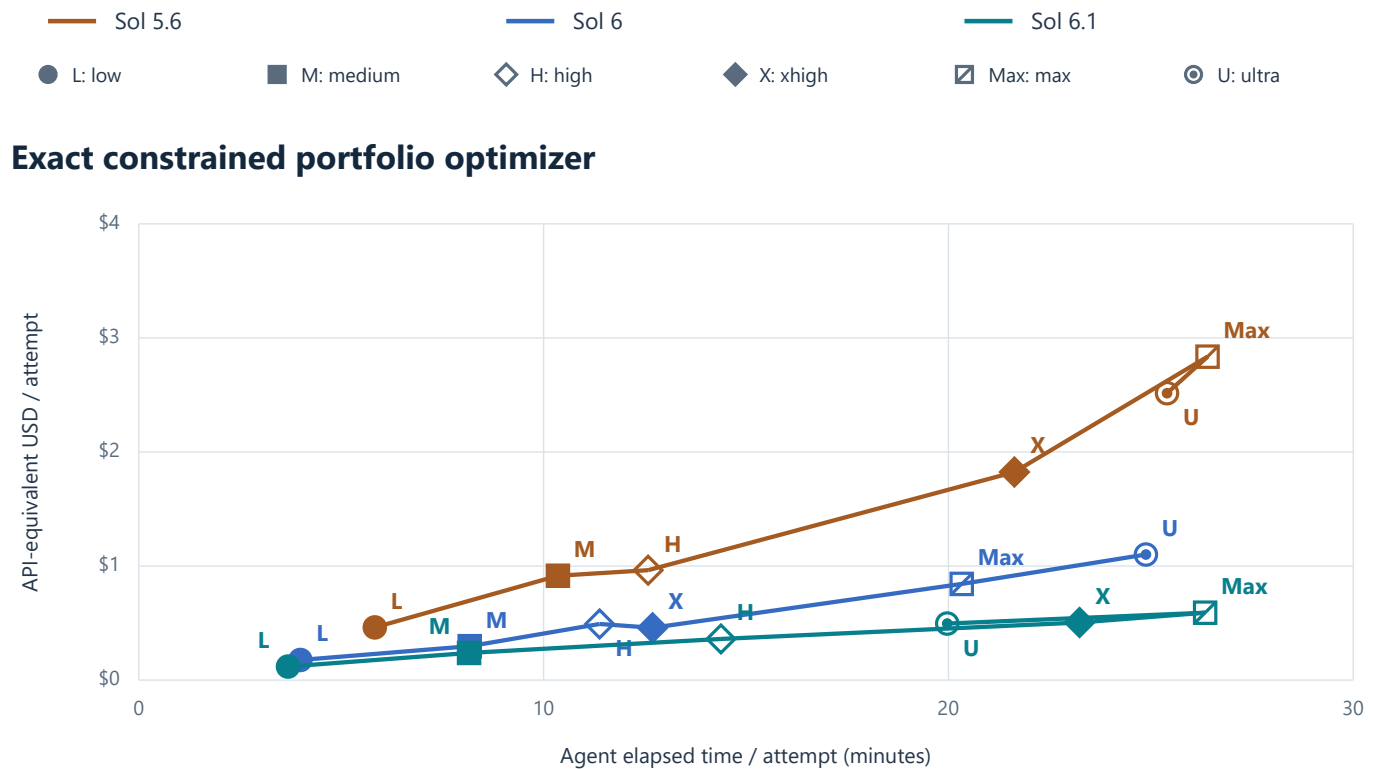
SQLite service



Intervals test inclusive union and purity; the cache tests TTL/LRU semantics; the DAG tests concurrency and failure propagation; the service combines SQLite transactions, HTTP and CLI behavior.

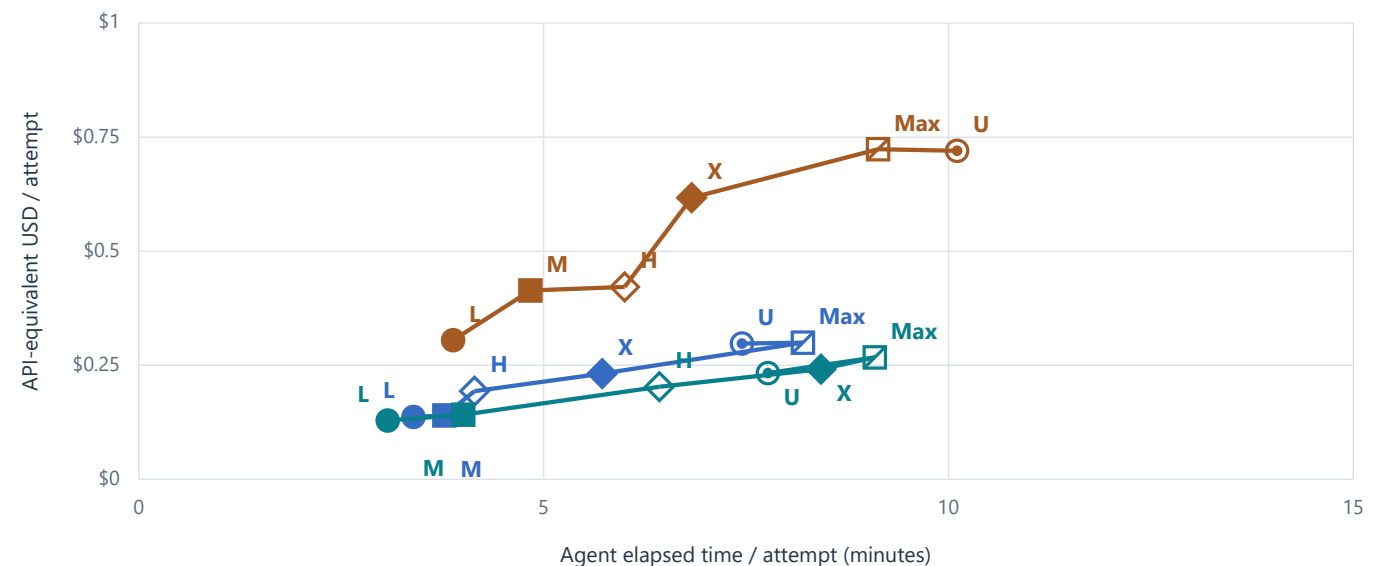
Workload trajectories: complex tasks

Algorithmic optimization and interacting transaction invariants add complementary complexity.



Signed values, transitive prerequisites, one-sided exclusions, mandatory projects, up to three resource budgets, exact optimality and cost/ID tie-breaking. Structured cases contain 32 projects.

Serializable MVCC store with recovery



Snapshot reads, staged writes, conservative optimistic validation, missing-key/range reads, savepoints, retained read history and checkpoint/log replay. Calls use deterministic in-memory interleavings.

Task-level dollars and seconds

Each cell shows mean API USD above mean elapsed seconds, across two repetitions.

Model / effort	Intervals	Cache	DAG	Service	Optimizer	MVCC
Sol 5.6 / low	\$0.160 69 s	\$0.210 108 s	\$0.220 125 s	\$0.554 280 s	\$0.462 350 s	\$0.305 233 s
Sol 5.6 / medium	\$0.200 91 s	\$0.250 130 s	\$0.279 165 s	\$0.856 399 s	\$0.915 621 s	\$0.414 290 s
Sol 5.6 / high	\$0.196 108 s	\$0.328 148 s	\$0.336 198 s	\$1.149 531 s	\$0.964 755 s	\$0.422 360 s
Sol 5.6 / xhigh	\$0.259 147 s	\$0.243 141 s	\$0.383 215 s	\$1.610 692 s	\$1.824 1,298 s	\$0.617 410 s
Sol 5.6 / max	\$0.309 179 s	\$0.350 212 s	\$0.484 305 s	\$1.925 908 s	\$2.836 1,584 s	\$0.724 548 s
Sol 5.6 / ultra	\$0.244 151 s	\$0.362 225 s	\$0.545 314 s	\$2.021 941 s	\$2.515 1,524 s	\$0.720 606 s
Sol 6 / low	\$0.046 68 s	\$0.086 78 s	\$0.113 110 s	\$0.191 253 s	\$0.176 240 s	\$0.137 204 s
Sol 6 / medium	\$0.078 88 s	\$0.118 112 s	\$0.117 145 s	\$0.310 299 s	\$0.297 490 s	\$0.140 226 s
Sol 6 / high	\$0.078 77 s	\$0.098 119 s	\$0.138 168 s	\$0.515 505 s	\$0.492 683 s	\$0.193 249 s
Sol 6 / xhigh	\$0.102 194 s	\$0.140 242 s	\$0.184 209 s	\$0.750 666 s	\$0.459 762 s	\$0.232 343 s
Sol 6 / max	\$0.157 219 s	\$0.140 275 s	\$0.202 301 s	≥ \$0.754* 2,380 s	\$0.843 1,220 s	\$0.300 492 s
Sol 6 / ultra	≥ \$0.082* 158 s	\$0.155 215 s	\$0.179 215 s	\$0.861 1,230 s	\$1.101 1,493 s	\$0.298 447 s
Sol 6.1 / low	\$0.050 72 s	\$0.058 96 s	\$0.079 123 s	\$0.169 272 s	\$0.118 221 s	\$0.129 184 s
Sol 6.1 / medium	\$0.066 81 s	\$0.073 105 s	\$0.082 152 s	\$0.245 413 s	\$0.238 490 s	\$0.142 240 s
Sol 6.1 / high	\$0.055 165 s	\$0.070 166 s	\$0.099 268 s	\$0.331 871 s	\$0.361 863 s	\$0.203 386 s
Sol 6.1 / xhigh	\$0.069 121 s	\$0.100 158 s	\$0.148 325 s	\$0.480 1,028 s	\$0.505 1,394 s	\$0.241 506 s
Sol 6.1 / max	\$0.118 230 s	\$0.117 224 s	\$0.146 385 s	\$0.509 1,096 s	\$0.591 1,580 s	\$0.267 545 s
Sol 6.1 / ultra	\$0.097 134 s	\$0.095 176 s	\$0.167 420 s	\$0.404 1,068 s	\$0.496 1,198 s	\$0.233 466 s

Stars mark partial cost telemetry. Agent time and token spending include self-tests and environment recovery during the attempt. Task limits are 10 / 20 / 40 / 40 / 40 / 40 minutes.

Complex reasoning: behavioral evidence

Every small optimizer oracle and every MVCC check passed. Only one structured optimizer scale check failed.

Model / effort	Opt V	Opt C	Opt O	Opt S	MV Sem	MV Iso	MV Rec	MV V
Sol 5.6 / low	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / medium	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / high	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / xhigh	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 5.6 / ultra	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / low	100.0	100.0	100.0	83.3	100.0	100.0	100.0	100.0
Sol 6 / medium	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / high	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / xhigh	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6 / ultra	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / low	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / medium	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / high	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / xhigh	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / max	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Sol 6.1 / ultra	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

Optimizer: validation 20%, constraints 25%, optimality 35%, scale 20%. MVCC: semantics 25%, isolation 35%, recovery 25%, validation 15%. Values are category percentages; subcases fail their containing check.

The optimizer uses an independent exhaustive oracle on 64 seeded small problems per attempt, greedy traps, all tie rules and three structured scale fixtures. MVCC uses 480-step mixed transaction traces, 180-transaction replay traces, every checkpoint/log prefix, ABA, phantoms, write skew and failed-operation atomicity.

Scale failure: Sol 6 / Low, optimizer repetition 1, exceeded 8 seconds on the positive-bound fixture: 31 affordable projects and one individually infeasible high-value decoy. Its task score is 93.33; the paired task mean is 96.67. All its exact small-case answers passed.

A separate bounded replay reproduced the 8-second timeout. The positive control and Low repetition 2 completed the same fixture in about 0.20 and 0.22 seconds. The original grades were retained.

These tasks distinguish tested correctness and scaling behavior. Near-perfect outcomes across most settings leave too little separation to claim which model is generally smarter.

Expanded correctness and found defects

The first four tasks retain the same quality-v2.1 checks and scores. Completed solutions pass 920/922 expanded checks.

Model / effort	Intervals	TTL/LRU cache	Concurrent DAG	SQLite service
Sol 5.6 / low	100.00	94.64	100.00	100.00
Sol 5.6 / medium	100.00	100.00	100.00	100.00
Sol 5.6 / high	100.00	100.00	100.00	100.00
Sol 5.6 / xhigh	100.00	100.00	100.00	100.00
Sol 5.6 / max	100.00	100.00	100.00	100.00
Sol 5.6 / ultra	100.00	100.00	100.00	100.00
Sol 6 / low	100.00	94.64	100.00	100.00
Sol 6 / medium	100.00	100.00	100.00	100.00
Sol 6 / high	100.00	100.00	100.00	100.00
Sol 6 / xhigh	100.00	100.00	100.00	100.00
Sol 6 / max	100.00	100.00	100.00	N/A
Sol 6 / ultra	N/A	100.00	100.00	100.00
Sol 6.1 / low	100.00	100.00	100.00	100.00
Sol 6.1 / medium	100.00	100.00	100.00	100.00
Sol 6.1 / high	100.00	100.00	100.00	100.00
Sol 6.1 / xhigh	100.00	100.00	100.00	100.00
Sol 6.1 / max	100.00	100.00	100.00	100.00
Sol 6.1 / ultra	100.00	100.00	100.00	100.00

Boundary defects: Sol 5.6 / Low r2 and Sol 6 / Low r1 cache constructors fail on a finite integer TTL of 10**400 with an integer clock. Applying math.isfinite to that integer raises OverflowError. Their original acceptance tests passed.

Expanded tests add 1,200 random unions, cache model traces, DAG failure graphs and concurrency barriers, plus reservation stock-model steps, races and persistence/error cases. Properties/boundaries/concurrency weigh 40/30/30, renormalized when a task lacks a category.

N/A is incomplete completed-code coverage. The provider-failed stub and saved timeout code remain in raw evidence; they are excluded from completed-code means. The new tasks use their own frozen behavioral rubric, without retroactively changing old scores.

Do the added tests detect defects?

Per-task mutation sensitivity, averaged across two completed candidates. N/A shows usable-score coverage for that pair.

Model / effort	Intervals	Cache	DAG	Service	Optimizer	MVCC
Sol 5.6 / low	50%	50%	70%	58%	0%	0%
Sol 5.6 / medium	100%	60%	70%	0%	0%	50%
Sol 5.6 / high	100%	60%	90%	0%	N/A (1/2)	42%
Sol 5.6 / xhigh	50%	60%	80%	0%	90%	42%
Sol 5.6 / max	50%	60%	100%	0%	90%	67%
Sol 5.6 / ultra	50%	60%	N/A (1/2)	0%	N/A (1/2)	58%
Sol 6 / low	0%	30%	70%	0%	40%	25%
Sol 6 / medium	88%	60%	100%	N/A (1/2)	N/A (0/2)	75%
Sol 6 / high	88%	60%	100%	N/A (0/2)	100%	67%
Sol 6 / xhigh	100%	N/A (0/2)	100%	N/A (0/2)	90%	83%
Sol 6 / max	100%	N/A (1/2)	100%	N/A (0/2)	90%	N/A (1/2)
Sol 6 / ultra	N/A (1/2)	70%	100%	N/A (0/2)	N/A (1/2)	83%
Sol 6.1 / low	100%	70%	80%	100%	N/A (1/2)	75%
Sol 6.1 / medium	100%	80%	100%	N/A (0/2)	N/A (0/2)	92%
Sol 6.1 / high	100%	80%	100%	N/A (0/2)	N/A (0/2)	100%
Sol 6.1 / xhigh	100%	80%	100%	N/A (0/2)	N/A (0/2)	100%
Sol 6.1 / max	100%	80%	100%	N/A (0/2)	N/A (0/2)	100%
Sol 6.1 / ultra	100%	90%	100%	N/A (0/2)	N/A (0/2)	100%

175/214 completed candidates have usable scores; 29 have no recognized added tests. On the complex tasks, 55/72 scores are usable and 10 candidates have no recognized added tests.

Discovery uses unittest-compatible tests/test*.py, excluding supplied test_public.py. Tests must pass on both candidate and positive control. No added tests score zero only for this criterion. Import errors, skips, timeouts and reference incompatibility are inconclusive.

Optimizer test strength is unavailable for 16/36 candidates: twelve positive-control test runs hit the 30-second timeout, three encounter import/execution issues, and one mutant run is inconclusive. These evaluator limitations particularly affect Sol 6.1 optimizer coverage and do not establish a candidate-code defect.

Fixed defect sets: 4 interval, 5 cache, 5 DAG, 6 reservation, 5 optimizer and 6 MVCC mutants. Contract-assertion failures detect a mutant; the new-task policy treats execution errors/hangs as unavailable. These bounded sets support per-task interpretation.

Generated-code runtime and allocation

Existing quality-v2.1 runtime / Python-allocation measurements for the first four tasks. Each cell is milliseconds / KiB.

Model / effort	Intervals	TTL/LRU cache	Concurrent DAG	SQLite service
Sol 5.6 / low	17.90 / 4,336	34.68 / 104	54.16 / 953	515.90 / 4
Sol 5.6 / medium	16.34 / 4,454	34.95 / 104	61.10 / 901	478.92 / 4
Sol 5.6 / high	18.84 / 4,454	35.75 / 104	63.61 / 870	533.39 / 4
Sol 5.6 / xhigh	17.02 / 4,454	35.53 / 104	60.55 / 877	553.25 / 5
Sol 5.6 / max	16.83 / 4,336	34.36 / 104	56.91 / 755	648.41 / 5
Sol 5.6 / ultra	17.61 / 4,336	33.78 / 104	53.81 / 863	611.56 / 4
Sol 6 / low	13.95 / 4,336	39.05 / 112	54.40 / 802	537.83 / 4
Sol 6 / medium	15.54 / 4,336	19.90 / 112	58.85 / 806	543.75 / 4
Sol 6 / high	15.16 / 4,336	40.23 / 112	55.15 / 730	693.05 / 4
Sol 6 / xhigh	16.97 / 4,336	40.44 / 112	51.28 / 813	622.31 / 4
Sol 6 / max	16.25 / 4,336	38.47 / 112	57.80 / 844	N/A
Sol 6 / ultra	N/A	36.34 / 108	57.56 / 838	504.32 / 4
Sol 6.1 / low	17.07 / 4,336	33.74 / 104	77.32 / 781	529.36 / 5
Sol 6.1 / medium	17.29 / 4,336	34.85 / 104	71.42 / 834	861.48 / 5
Sol 6.1 / high	16.68 / 4,336	35.37 / 104	66.49 / 806	555.97 / 5
Sol 6.1 / xhigh	16.82 / 4,336	35.33 / 104	68.93 / 730	468.35 / 5
Sol 6.1 / max	17.24 / 4,336	38.25 / 104	55.46 / 704	511.43 / 5
Sol 6.1 / ultra	17.30 / 4,336	34.98 / 104	52.31 / 705	467.98 / 5

One warm-up, then the median of three sequential runs. Workloads: 30,000 intervals; cache capacity 1,000 with 1,000 writes and 3,000 reads; 1,500 DAG nodes; 40 reservation cycles. Allocation is a separate tracemalloc measurement, not process RSS or native/SQLite memory.

These values describe execution after generation, separate from agent completion time. The complex tasks have their scored scale/trace checks; equivalent runtime and memory profiles were not collected for them. Small differences can reflect scheduling noise.

Repetition spread

Median second-to-first ratios across six task pairs. Two observations per pair support descriptive spread, not confidence intervals.

Model / effort	USD r2 / r1	Time r2 / r1	Largest time spread
Sol 5.6 / low	0.82	0.97	1.42x
Sol 5.6 / medium	0.82	0.98	1.43x
Sol 5.6 / high	1.11	1.09	1.47x
Sol 5.6 / xhigh	0.92	0.95	1.52x
Sol 5.6 / max	0.68	0.82	2.21x
Sol 5.6 / ultra	0.96	0.98	1.20x
Sol 6 / low	0.68	0.77	1.57x
Sol 6 / medium	0.93	0.95	1.45x
Sol 6 / high	1.04	1.08	1.22x
Sol 6 / xhigh	0.97	1.03	2.24x
Sol 6 / max	N/A	1.00	1.31x
Sol 6 / ultra	N/A	1.05	1.88x
Sol 6.1 / low	0.96	1.10	1.25x
Sol 6.1 / medium	0.94	0.92	1.87x
Sol 6.1 / high	0.97	0.78	1.68x
Sol 6.1 / xhigh	0.81	1.00	1.80x
Sol 6.1 / max	1.12	0.97	1.57x
Sol 6.1 / ultra	0.87	0.90	1.63x

A ratio above 1 means repetition two used more money/time. The last column is the largest max/min time ratio among that setting's six pairs, including interruptions. Server load and tool behavior can affect elapsed time.

Paid attempts are sequential, with model/effort order interleaved and reversed inside each task in repetition two. There are no automatic retries, model fallbacks or post-grading feedback.

Code quality and design review

Behavior, test strength and execution efficiency are reported separately. Manual rubric scoring remains pending.

Evidence layer	Scope	Interpretation
Main functional score	All six tasks	Equal task weights; fixed category weights for the complex tasks.
Expanded correctness	First four tasks	Frozen properties, boundaries and concurrency checks.
Own-test strength	All six tasks	Sensitivity to fixed per-task contract mutations.
Runtime / memory	First four tasks	Descriptive generated-code measurements, unscored.
Manual design rubric	Pending	Clarity, design and failure maintenance, with source/line evidence.

Manual rubric

Area	Evidence a reviewer should inspect
Clarity	Intent, names, coherent control flow, explanations and useful tests.
Design	Interfaces, responsibilities, cohesion, duplication and contract fit.
Failure maintenance	Error handling, cleanup, transactions, concurrency and safe changes.

The existing four-task quality formula uses 30% original correctness, 35% expanded correctness, 20% own-test strength and 15% manual review. Its normalized 85-point automatic evidence is not a full quality score. It is preserved for those tasks and is not extrapolated to the added tasks.

Static line/function counts are review signals, not grades. Tests tied to private implementation details can be valid for the candidate while yielding inconclusive cross-implementation mutation evidence.

Method, token prices and reproducibility

Each attempt uses one turn, a fresh session/workspace, the exact requested model/effort and Standard speed. Task prompt hashes are identical within each task across all 36 attempts. SHA-256 manifests protect the prepared inputs and saved candidate code.

Model	Input	Cached input	Cache write	Output
Sol 5.6	\$4.00	\$0.40	\$5.00	\$20.00
Sol 6	\$2.00	\$0.20	\$2.50	\$10.00
Sol 6.1	\$2.00	\$0.10	\$2.50	\$10.00

Rates are USD per one million tokens, frozen on 30 Sep 2026 and verified on 1 Oct. The same schedule is used throughout for comparability. This report does not assert a newly checked current price. Ultra is a verified Codex effort setting; the USD calculation does not imply an API Ultra option.

Uncached input, cached reads, cache writes and output are priced without double counting. Reasoning tokens are part of output. Per-request input above 272,000 tokens applies the frozen 2x input/cache and 1.5x output multipliers. Hosted-tool fees and regional premiums are excluded.

Exact token estimates: 214/216. Observed total is at least \$87.59; two partial-usage attempts prevent a complete total. The two complex tasks have exact observed token cost \$41.81 across 72 attempts.

Secondary token-based credit estimate: 2,189.95 observed credits, including partial usage. Shared account balance changes do not identify billed credits for an individual run.

Pinned runtime: codex-cli 0.159.0; Node v22.18.0; Python 3.14.0 on Windows. Shell/tool recovery is included in agent time. Read isolation is instruction-based; automated command scans are diagnostics.

Sources and evidence

Sol 5.6 API rates | Sol 6 API rates | Sol 6.1 API rates
Codex credits | Codex app-server | Chart presentation reference

Local evidence: analysis/results.json and comparison.csv; analysis/complex-test-strength/results.json; analysis/report_audit.json; preparation/freeze.json; the preserved quality-v2.1 results; per-run candidate snapshots, prompt.md and rpc.jsonl. All 216 snapshots/settings/turns/price calculations were audited.

Results SHA-256: efe52197a02f4c255733d7bd9911d9088fbde54204ee870ea0d36e51483fc740
Quality SHA-256: beec8746f0ab9677df1f850738f2f5f3ddc147084435225309ff9998407477c0
Test-strength SHA-256: c3b85878d924e78133d8a8cf1e735aee3f13deebd7015fdeb06eb4c5b783c4dd